

COMPUTER ORGANISATION AND DESIGN

UCSH-301

Name:- G. Vardhan

Regd. No.:- 234211

Class:- 2nd B.S.

Addressing Modes

H. Vardhan 20/20

Addressing modes define how the CPU identifies the operands for instructions during program execution.

They are essential for determining the source or destination of data, whether it is in memory, in a register, or part of the instruction itself.

→ Types of Addressing Modes:

- Immediate
- Direct
- Indirect
- Register
- Register indirect
- Displacement
- Stack.

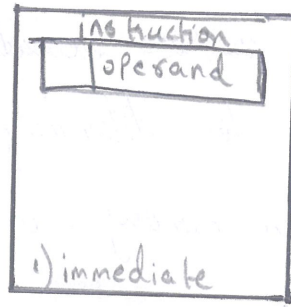
Note: When illustrating the addressing modes we use the following notations:

- A = Contents of an address field in the instruction
- R = Contents of an address field in the instruction that refers to a register
- EA = actual (effective) address of the location containing the referenced operand.
- (X) = contents of memory location X or register X.

1) Immediate Addressing Mode:

The simplest form of addressing is immediate addressing.

In this type, the operand value is directly present in the instruction.



$$\text{Operand} = A$$

Principal Advantage:- No memory reference.

Principal Disadvantage:- Limited operand magnitude.

- It is used to define and use constants or set initial values of variables.
- The number is stored in 2's complement form.
- Left most bit serves as a sign bit.

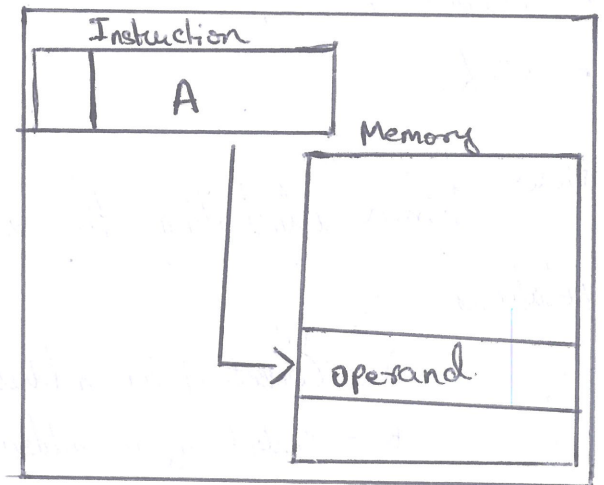
2) Direct Addressing mode:-

• The memory address of the operand is directly specified in the instruction.

• CPU fetches the data from the specified memory location.

• Advantage: It requires only one memory reference and no special calculation. Saves time and storage.

• Disadvantage: Provides only limited address space.



Direct addressing

$$EA = A$$

Eg:- LOA 2042H load the value from memory address 2042H into accumulator.

→ Indirect Addressing mode:-

- $EA = (A)$
- The address of the operand is specified by a register pair (such as HL or DE)
- The register pair holds the address where the data is stored.

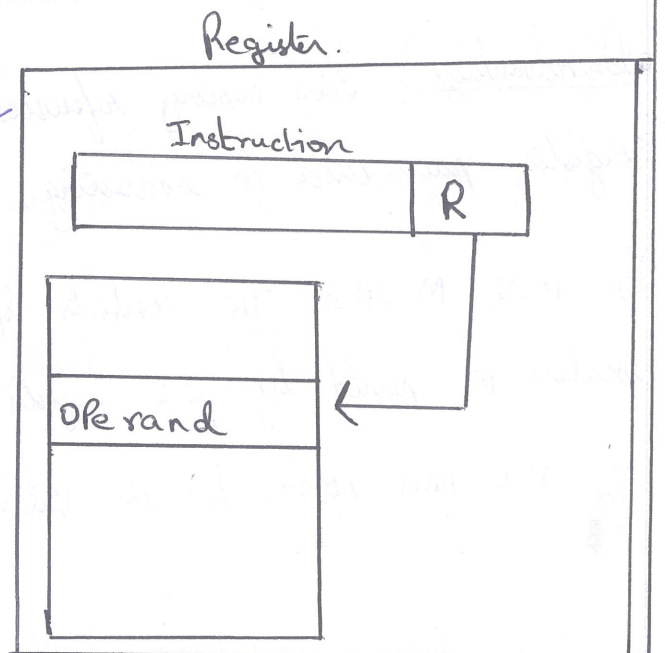
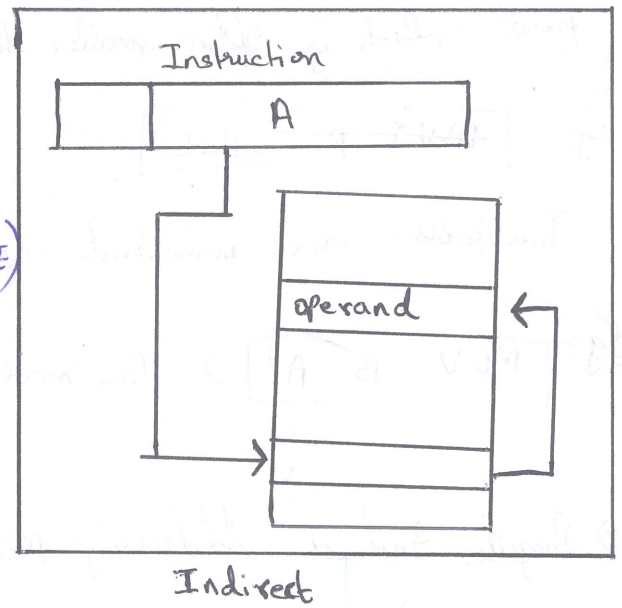
Advantage: For a word length of 'n' an address space of 2^n is available.

Disadvantage: The instruction execution requires two or more memory references to fetch and to get the address of the operand and to get its value. It takes more time.

Eg:- MOV A M ⇒ Here, the contents of the memory location is pointed by HL pair is moved to accumulator.

⇒ Register Addressing:-

- $EA = R$
- The operand is in a register and the address of the register is present in the instruction.
- The operand is in one of the registers.
- Immediate binary value is interpreted as unsigned non-negative integer.
- Advantage: Only the instruction fetch is needed which saves memory or cache cycle & time.



- Disadvantage: The operand size is restricted by instruction's address field, which is often smaller than word length.

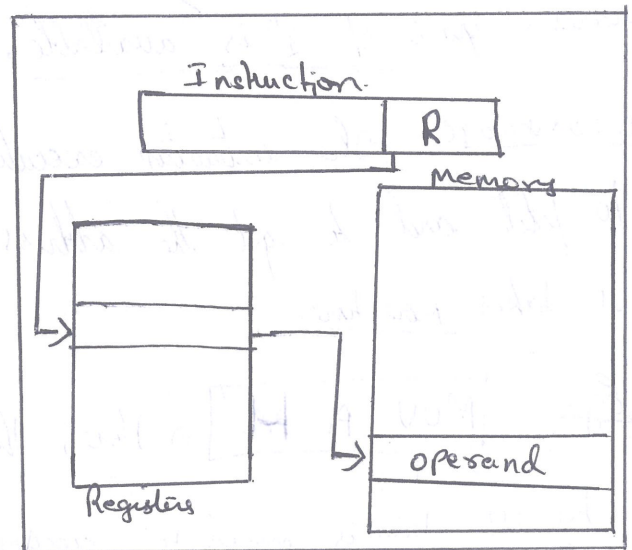
Eg: ~~`MOV A 0AH`~~

This tells ~~move immediate value 0A into register A.~~

Eg: `MOV B A` $\Rightarrow$ This moves the value in A to B.

$\Rightarrow$ Register Indirect Addressing Mode:-

- $EA = (R)$
- It is analogous to indirect addressing.
- In the instruction contains register and in that register, that tells where operand is present in the register or in the main memory.



Register Indirect.

Advantage: It has more memory space for fetching the operand. The instruction itself does not need to store memory address.

• Disadvantage: More memory references are done to fetch data and limited register pairs used for addressing.

Eg: `MOV M, A` $\Rightarrow$ The contents of register A is stored in memory location M pointed by H-L register pair.

The H-L pair points to the value which is stored in A.

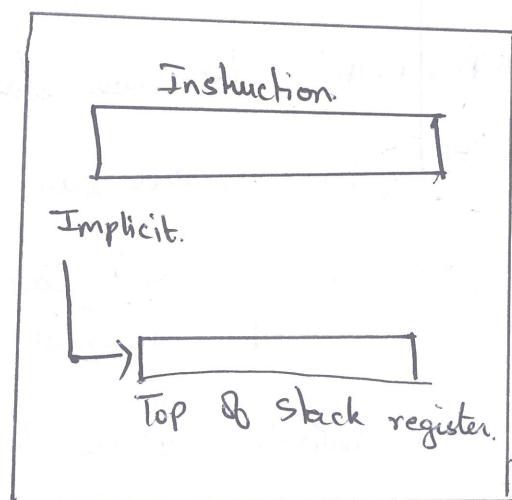
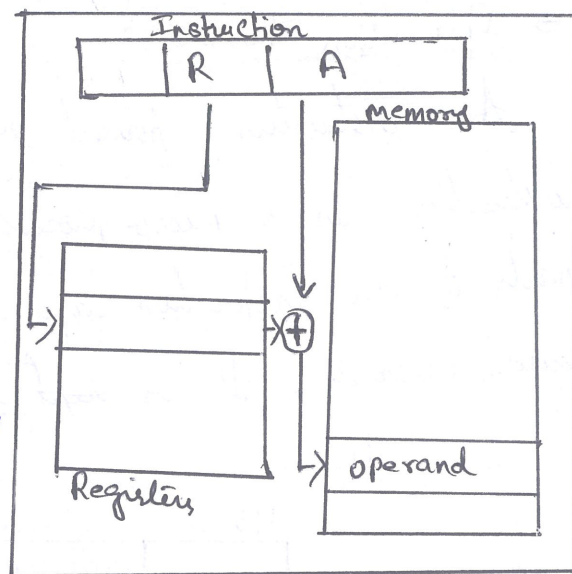
→ Displacement Addressing Mode:-

- $EA = A + (R)$
- It combines capabilities of direct and register indirect addressing modes.
- This mode consists 2 address fields, at least one of which is explicit.
- The value contained in one address field (value=A) is directly used and contents at address field (value=R) refers to a register. These two addresses are added to produce effective address value where the operand is located.
- Advantage : It has same advantages as direct and register indirect modes of addressing. It is flexible to fetch data.

• Disadvantage:- It is more complex in nature compared to all other addressing modes.

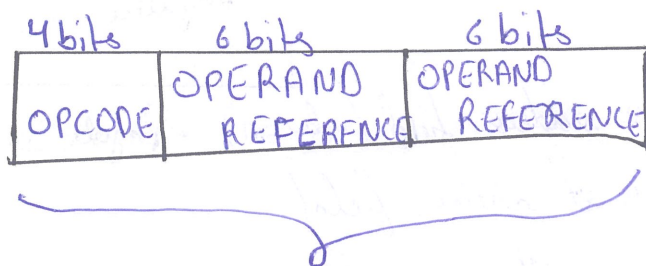
* Stack Addressing :-

- $EA = \text{top of stack}$.
- Data is stored and retrieved from the stack using the stack pointer.
- Advantage:- There is no need for memory reference. It is efficient and useful for handling function calls, local variable & interrupts.
- Disadvantage:- It is limited by stack size, so many push operation can cause stack overflow.



→ Instruction Format :-

An instruction format refers to the structure or layout of an instruction in a micro processor. The format specifies how different parts of an instruction are arranged such as operation code (opcode) and operands. It is ~~rep~~ represented by a sequence of bits.



16-Bits.

- Instruction Format : [8085 Micro-processor (16 bit)]

Elements of Instructions:-

- **OPCODE** : Defines the operation to be performed, such as ADD, SUB, MOV etc.
- **OPERANDS** : The main data on which the operation is performed.
 - **Registers** : operands can reside in CPU registers.
 - **Memory** : Operands can be stored in memory location which can be accessed by the address locations.
 - **Immediate values** : It can be constant value, directly embedded in the instruction such as 98H.
- **Addressing Mode** :- Describes how to access the operands such as direct, register, indirect etc.
- **Instruction length** : It has a certain length which determines how much

information they contain, it differs from processors.

• Flags:- Some instructions may set or clear flags in the status register depending on the result of the operation.

→ Operand types:-

Operands are the data items that are processed by instructions.

The types of operands are:

→ Numbers:- All machine languages include numeric data types.

→ Characters:- A common form of data is text or character strings.

→ Logical Data:- Each word or other addressable unit (byte, half word) is treated as a single unit of data.

⇒ Instruction Cycle:-

It is the process a CPU follows to execute an instruction.

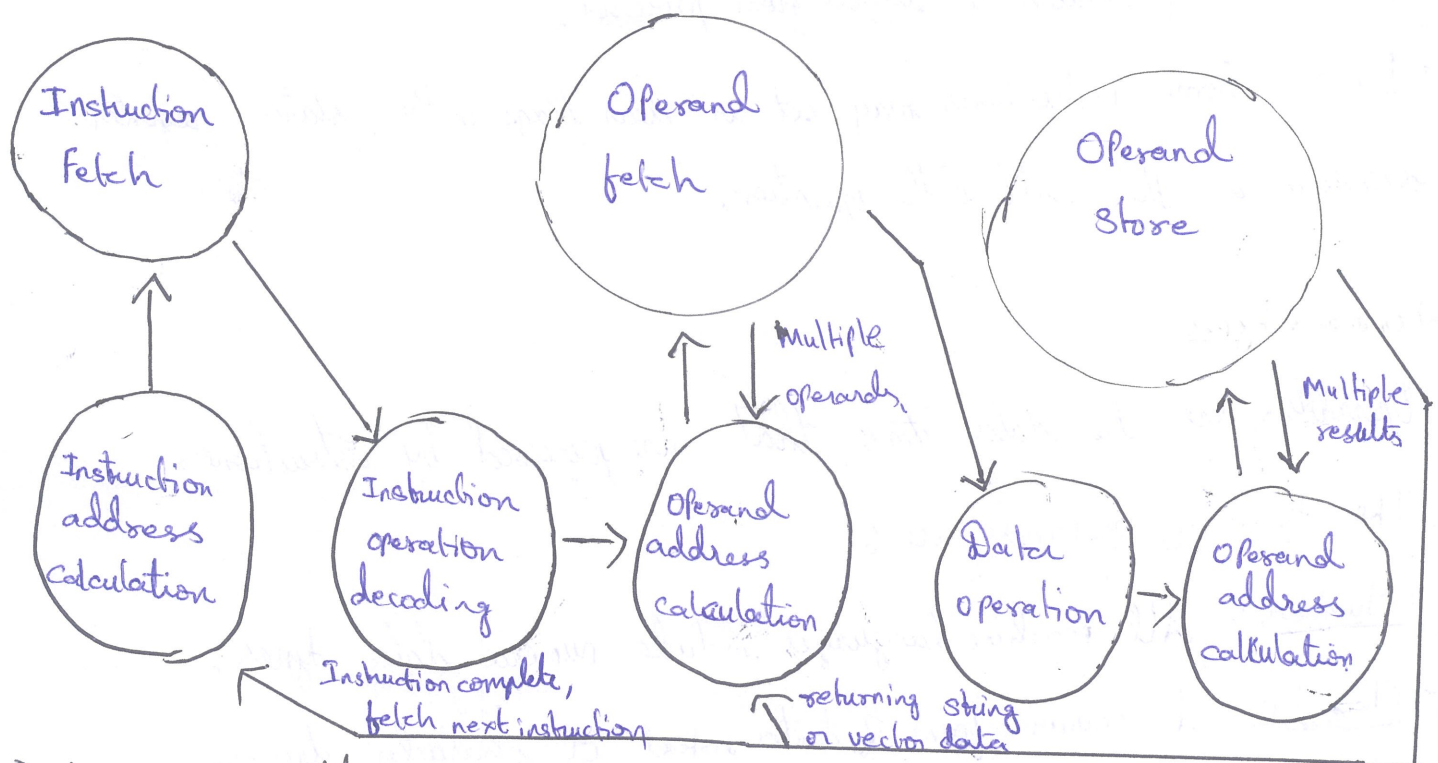
It consists of four main steps:-

1. Fetch:- The CPU retrieves the instruction from memory, based on the address in the program counter.

2. Decode:- The CPU decodes the fetched instruction to understand what operation to perform and identifies the operand.

3. Execute:- The CPU performs the operation specified by the instruction.

4. Interrupt:- If interrupts are enabled and an interrupt has occurred, save the current process state and service the interrupt.



⇒ Number of Address :-

It refers to how many operands are specified in an instruction. The number of addresses determines how many values the instruction can operate on at once.

→ I address instructions:- Specifies one operand and the other operand is implicitly assumed to be the accumulator.

Eg:- According to 8085 processor.

<u>Instruction</u>	<u>Comment</u>	<u>Instruction</u>	<u>Comment</u>
1) LOAD D	$\text{AC} = \text{D}$	5) LOAD A	$\text{AC} = \text{A}$
2) MPY E	$\text{AC} = \text{AC} \times \text{E}$	6) SUB B	$\text{AC} = \text{AC} - \text{B}$
3) ADD C	$\text{AC} = \text{AC} + \text{C}$	7) DIV Y	$\text{AC} = \text{AC} \div \text{Y}$
4) STOR Y	$\text{Y} = \text{AC}$	8) STOR Y	

2-address instruction :-

These instructions specify 2 operands and source and one destination.

The result is stored in destination.

- These are more flexible, since both source & destination operands are specified.
- One operand is overwritten by result.

Eg: according to 8085 processor

Instruction	Comment	Instruction	Comment
1) MOVE VA	$V = A$	4) MPY TE	$T = T * E$
2) SUB VB	$V = V - B$	5) ADD TC	$T = T + C$
3) MOVE TD	$T = D$	6) DIV T	$V = V \div T$

3-Address instruction:-

These instructions specify 3 operands - 2 source operands and one destination operand, operation is performed on 2 source operands and result is stored in destination operand.

- These are very flexible, allows specifying separate operands for the sources and destination.

Eg: 8085 processor

Instruction - Comment

* Instruction types:-

Refers to the classification of instructions based on the kind of operations performed.

- 1) Dataprocessing:- These instructions handle logical and arithmetic operations to process data.
- 2) Data movement:- These instructions move data between registers, memory, and I/O devices.
- 3) Data storage:- These instructions are used to store or retrieve data from memory.
- 4) Data control:- These instructions control the operations of the microprocessor itself, such as setting or checking flags, interrupts, and jumps.

* Instruction set Design:-

Defines the instructions a CPU can execute and how they operate.

Consists of:-

- 1) operation repertoire
- 2) Data types
- 3) Instruction format
- 4) Addressing

Flags

1. Status register or flag register stores the flags that reflect the outcome of the last arithmetic or logical operation.

These flags indicate the status of the processor and help determine the next course of action in a program, such as

sign flag :- (S)

The sign flag is set (1) if the result of an operation is negative, else 0.

Eg:- After subtracting a larger no. from a smaller no. (eg. 5-10), the result is negative, so sign flag will be set.

zero flag :- (Z)

The zero flag is set (1) if the result of an operation is zero.

Eg:- After the operation $\boxed{A = A - A}$ (where A is accumulator), the zero flag is 1.

Auxiliary Carry flag :- (AC)

The auxiliary carry flag is set (1) if there is a carry from bit 3 to bit 4 during an arithmetic operation, especially useful for BCD (Binary Coded Decimal) arithmetic.

$$\begin{array}{r} \text{Eg:- } 0000'1001 \quad [09H] \\ + 00000111 \quad [07H] \\ \hline 00010000 \end{array}$$

Parity flag :- (P)

The parity flag is set if the result of an operation has an even no. of 1s.

Eg:- if the result of an operation is 10101010 (has 4 1s) the Parity flag is set 1.

Carry flag :- (CY)

The carry flag is set (1) if there is a carry out of the most significant bit in an arithmetic operation.

Eg:-

$$\begin{array}{r} 10100000 \\ + 01100000 \\ \hline 10000000 \end{array}$$

Carry $\rightarrow$ So, the carry flag is set 1.

Structure of the Flag Register

D7	D6	D5	D4	D3	D2	D1	D0
Sign	Zero	Unused	Auxiliary	Unused	Parity	Unused	Carry
S	Z	-	AC	-	P	-	CY

Explanations

Instruction set diagram:

It represents how an instruction is formatted in a microprocessor, like 8085. Each instruction consists of 1 to 2 or 3 bytes long.

Instruction Types:-

- 1- byte instructions:- Only contains an opcode (eg. NOP, INR A)
- 2- byte instructions:- Contains an opcode and one operand (eg. MOV A, 05h)
- 3- byte instructions:- Contains an opcode and two operands (eg. LXI H, 2050h)

Registers:-

Registers are small, fast storage locations within a microprocessor used to hold data temporarily during processing.

Types of Registers:-

- 1) Accumulator (AC): A special register used for arithmetic and logical problems.
 - Most operations are performed directly on the Accumulator.
- 2) General purpose Registers:-
 - In (8085P) there are six 8-bit general-purpose registers:
B, C, D, E, H, L.

4. Stack pointer :- (SP)

- A 16-bit register that points to the current top of the stack in memory.
- Stores return addresses and local variables during subroutines calls.
(interrupts)

5) Instruction Register (IR)

- Holds the currently executing instruction
- It is part of the control unit and it decodes the instruction